



OBJECT ORIENTED PROGRAMMING (23EN1202)

UNIT-5: INHERITANCE IN PYTHON

Syllabus



INHERITANCE:

Introduction, inherit classes in python, Polymorphism and method overriding, Types of inheritance

(Text Book: Chapter 10: 10.1 to 10.3)

What is Inheritance?



Definition: Inheritance allows a class (child) to **acquire properties and methods** of another class (parent).

Why Inheritance?

- Code reusability
- Avoids duplication
- Adds extensibility
- Supports polymorphism

Basic Terms

- Parent/Super class
- Child/Sub class

Python Inheritance Syntax

class BaseClass:

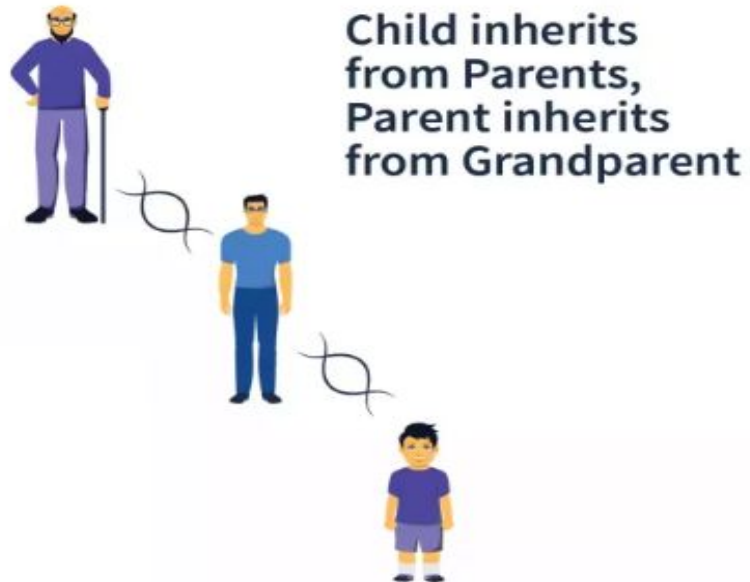
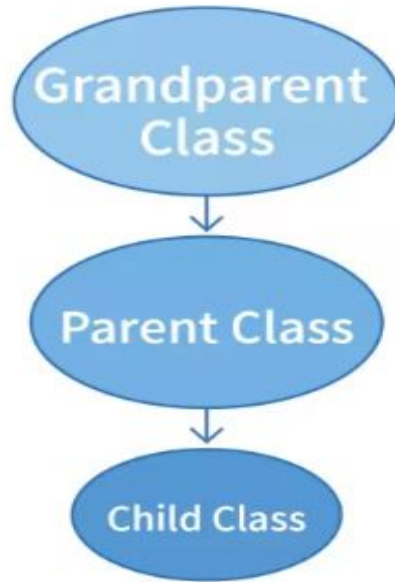
Body of base class

class DerivedClass(BaseClass):

Body of derived class

Derived class inherits features from the base class, adding new features to it. This results into re-usability of code.

Inheritance



Derived Class / Child Class



A derived class:

- Inherits all methods and variables of parent
- Can add **new features**
- Can **override** parent methods

Syntax

```
class Child(Parent):  
    # new methods or overridden methods
```

Constructor in Inheritance

 Parent constructor is NOT automatically called.

We must call it using **super()**.

```
class Parent:
```

```
    def __init__(self):
```

```
        print("Parent Constructor")
```

```
class Child(Parent):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        print("Child Constructor")
```

```
c = Child()
```

Method Overriding



Child class **redefines** parent's method.

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

Dog().sound()
```


Types of Inheritance in Python



1. **Single Inheritance**
2. **Multilevel Inheritance**
3. **Multiple Inheritance**
4. **Hierarchical Inheritance**
5. **Hybrid Inheritance**

Single Inheritance

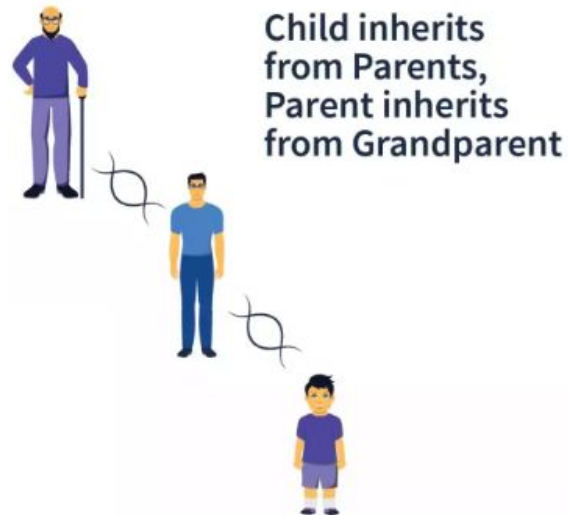
- In single inheritance, a subclass inherits from only one superclass.
- This is the simplest form of inheritance.
- One parent → one child.



Child
Inherits
qualities
from parent

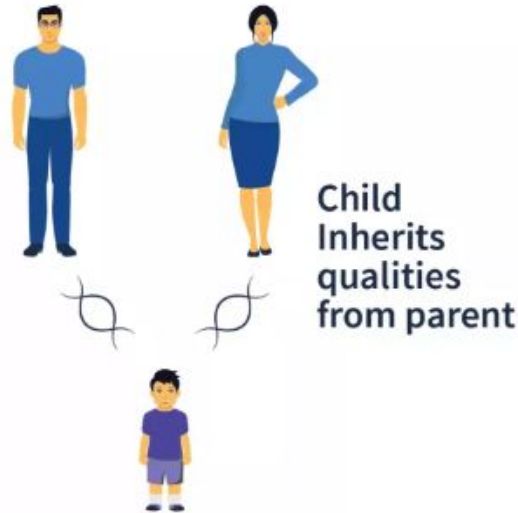
Multilevel Inheritance

- In multilevel inheritance, a subclass inherits from another subclass, forming a chain of inheritance.
- This creates a hierarchy of classes where each class inherits from the class above it.
- $A \rightarrow B \rightarrow C$



Multiple Inheritance

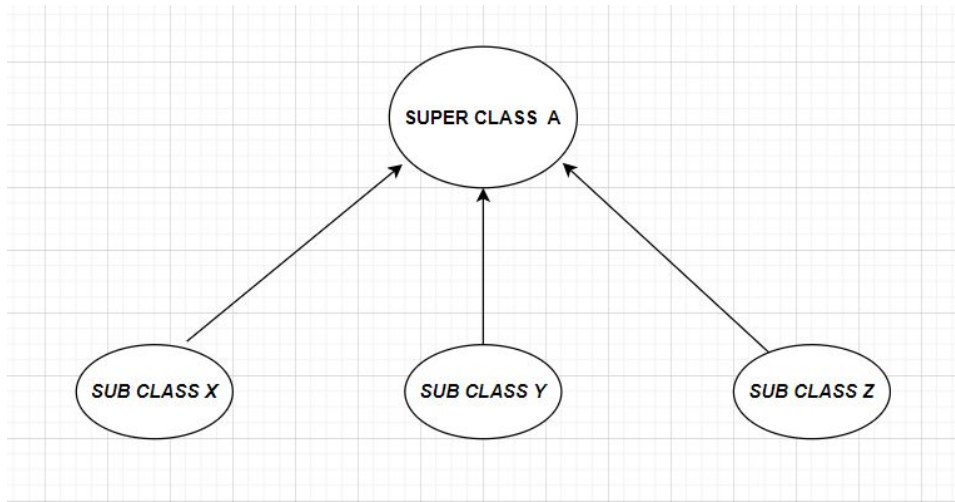
- In multiple inheritance, a subclass inherits from more than one superclass.
- This allows the subclass to access attributes and methods from multiple parent classes.



Hierarchical Inheritance



- One parent → multiple children.



Hybrid Inheritance

Hybrid Inheritance = Combination of two or more types of inheritance.

For example: **Common hybrid structure (Diamond shape):**



- **Multiple Inheritance** (D inherits from B and C)
- **Hierarchical Inheritance** (A is parent of B and C)
- **Multilevel Inheritance** ($A \rightarrow B \rightarrow D$ and $A \rightarrow C \rightarrow D$)

What is Polymorphism?



- Polymorphism , in simple terms, refers to having several different forms.
- It is one of the key features of OOP.
- Polymorphism is the ability of different objects to respond to the same message (method call) in different ways.
- In the context of object-oriented programming, this often means that different classes can define the same method name but with different implementations.

- In our example, imagine a picture of a person. This person can transform into various roles: a student, an employee, a customer, a family member, and a husband. Each role represents a different behavior or function of the person.
- Polymorphism simplifies code structure and promotes reusability by allowing objects to exhibit different behaviors while maintaining a common interface.



Polymorphism via Method Overriding



- Method overriding is the ability of a class to change the implementation of a method provided by one of its ancestors.
- It is an important concept of OOP since it exploits the power of inheritance and polymorphism.
- It allows the subclass to provide a specialized version of the method that is tailored to its own behavior while still maintaining the same method signature as the superclass.